Department of Computer Engineering

Faculty of Engineering, Kasetsart University

HW#5: SQL-III - Advanced SQL with Your Own Database

Continue Your HW#4 Database + Formulate Your Own SQL-III Tasks

After-Deadline Submissions Will Not Be Accepted

Objective

Continue using the relational database you designed in HW#4. Extend it only when necessary, then formulate your own meaningful SQL-III questions and SQL statements. The instructor specifies the SQL-III concept that each task must demonstrate; you decide what information is useful to retrieve or what database object/rule is useful to create.

SQL-III Scope

- Subquery in SELECT, including correlated subqueries
- Subquery in FROM (derived table)
- CASE expressions and CASE with aggregate functions
- Common Table Expression (CTE) using WITH
- VIEW: create, use, and drop a reusable virtual table
- MySQL Date/DateTime functions: CURRENT_DATE, CURRENT_TIMESTAMP, YEAR(), MONTH(), DAY(), DATEDIFF(), DATE_ADD()
- Database constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, and referential actions
- Indexes: CREATE INDEX, DROP INDEX, and choosing useful indexed columns

Part A: Reuse and Extend Your HW#4 Database

Use the same application domain and database from HW#4. You may add columns, records, or a new table if your existing design does not support one of the SQL-III concepts below.

- Keep at least 3 related tables with correct PRIMARY KEY and FOREIGN KEY constraints.
- Your database must contain at least one DATE or DATETIME attribute. Add one if necessary.
- Your data should have enough variation to make CASE, date, aggregate, and subquery results meaningful.
- Do not redesign the entire database unless necessary; this homework should extend the database you already created.

Database name: You may continue using hw4_STUDENT_ID, or make a copy named hw5_STUDENT_ID.

Part B: Formulate Your Own SQL-III Tasks

At least 5 tasks are required.

For each selected task, first write a meaningful question/purpose in English or Thai based on YOUR database, then write the SQL statement. Your SQL must visibly use the required concept.

Coverage rule: Your submitted work must include at least one task from Advanced Queries (Tasks 1-5), at least one from Reusable SQL (Tasks 6-7), and both Database Design/Performance tasks (Tasks 9-10).

Group A - Advanced Queries

Task 1 - Subquery in SELECT

Required concept: Subquery in SELECT

Create a query that shows detail rows together with one calculated summary value produced by a subquery in the SELECT clause. The SELECT subquery must return a single value for each output row.

Task 2 - Correlated Subquery

Required concept: Correlated Subquery

Create a query in which the inner query refers to a column from the current row of the outer query. A useful pattern is a per-row COUNT, SUM, AVG, or existence-related calculation.

Task 3 - Subquery in FROM

Required concept: Alias Table / Subquery in FROM

Create a multi-step query. The inner SELECT must create an intermediate result in the FROM clause, give the derived table an alias, and let the outer query further filter, aggregate, or summarize that result.

Task 4 - CASE Expression

Required concept: CASE

Create a meaningful classification or business rule using CASE, for example converting a numeric measure into categories such as High/Medium/Low. Your CASE expression must return a value.

Task 5 - CASE + Aggregate

Required concept: CASE + Aggregate + GROUP BY

Create a grouped summary using conditional counting or conditional summation, for example SUM(CASE WHEN ... THEN 1 ELSE 0 END). The query should produce a useful report from your own database.

Group B - Reusable / Readable SQL

Task 6 - Common Table Expression (CTE)

Required concept: WITH ... AS (...)

Create a CTE that gives a meaningful name to an intermediate query result, then query that CTE in the same SQL statement. The CTE should make the SQL easier to read than a deeply nested query.

Task 7 - VIEW

Required concept: CREATE VIEW + Use View

Create a reusable VIEW from a useful query in your database. Then write a second SELECT statement that queries the view. Use a meaningful view name. Include DROP VIEW only if you need to recreate it during testing.

Group C - Date / Time

Task 8 - Date / DateTime

Required concept: MySQL Date Functions

Create a meaningful query using at least TWO MySQL date/time functions from the lecture, such as YEAR(), MONTH(), DAY(), DATEDIFF(), CURRENT_DATE, CURRENT_TIMESTAMP, or DATE_ADD().

Part C: Database Rules and Performance

Tasks 9 and 10 are REQUIRED.

Task 9 - Constraints

Required concept: NOT NULL / UNIQUE / CHECK / DEFAULT / FOREIGN KEY

Add or create a table/column that demonstrates meaningful database rules. Use at least THREE constraints from NOT NULL, UNIQUE, CHECK, DEFAULT, and FOREIGN KEY. If using a FOREIGN KEY, you may also demonstrate ON DELETE or ON UPDATE behavior.

Task 10 - Index

Required concept: CREATE INDEX

Choose a non-primary-key column (or useful combination of columns) that is frequently searched, joined, filtered, or ordered in your database. Write a representative SELECT query, create an index that could support that access pattern, and briefly explain why you chose that column.

What query/access pattern is this index intended to support?

Submission Requirements

- Submit ONE file: HW5_Student_ID.sql
- Include any ALTER/CREATE/INSERT statements needed to extend your HW#4 database and all selected SQL-III tasks.
- **At least 6 tasks are required, following the coverage rule above; Tasks 9 and 10 are mandatory.**
- Before each task, write your own question/purpose as a SQL comment.
- Clearly label each task, for example /* Task 3 - Subquery in FROM */.
- For Task 7, include both CREATE VIEW and a SELECT that uses the view.
- For Task 10, include the SELECT query you want to support and the CREATE INDEX statement.
- The complete script must run in MySQL Workbench without manual correction.
- No screenshots or PDF are required unless specifically requested by the instructor or TA.

Suggested SQL File Format

```
/* Domain: _____________________________
   Reused from HW#4 */

USE hw4_STUDENT_ID;

/* Optional: extend schema/data for SQL-III */
ALTER TABLE ...;
INSERT INTO ...;

/* Task 1 - Subquery in SELECT
   My question: _____________________________________________ */
SELECT ...;

/* Task 6 - CTE */
WITH ... AS (...)
SELECT ...;

/* Task 9 - Constraints */
CREATE TABLE ...;

/* Task 10 - Index */
SELECT ...;
CREATE INDEX ... ON ... (...);
```

Checklist Before Submission

☐ I reused or logically extended my HW#4 database.

☐ My database contains a DATE/DATETIME attribute.

☐ I completed at least 6 tasks and followed the coverage rule.

☐ Tasks 9 (Constraints) and 10 (Index) are included.

☐ Each selected task uses the required SQL-III concept.

☐ My VIEW can be queried after it is created.

☐ My index choice is related to an actual query/access pattern.

☐ The complete .sql script runs successfully in MySQL Workbench.

Important Notes

- Use MySQL syntax.
- The emphasis is on choosing an appropriate SQL-III concept for a meaningful problem in your own database.
- For INDEX, remember that indexes can improve reads but also require storage and maintenance for INSERT/UPDATE/DELETE.